

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch: 2023-2028	Due date:

Experiment 4: Ensemble Prediction and Decision Tree Model Evaluation

Name: Mani Chidambaram

Reg No: 3122237001024

1. Aim and Objective

To build classifiers such as Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and Stacked Models, and evaluate their performance through hyperparameter tuning, 5-Fold Cross-Validation, ROC analysis, and feature importance interpretation.

2. Libraries Used

- **Pandas & NumPy:** For data loading, manipulation, and numerical operations.
- **Scikit-learn:** For data preprocessing, model implementation (Decision Tree, ensemble methods), hyperparameter tuning, and evaluation.
- **XGBoost:** For implementing the XGBoost classifier.
- **Matplotlib & Seaborn:** For data visualization, including confusion matrices, ROC curves, and feature importance plots.

3. Code Summary

The Python script implements the complete workflow: loading and preprocessing the dataset, training Decision Tree and ensemble models, applying hyperparameter tuning with GridSearchCV, and evaluating model performance.

```

1 # 1. Load and Preprocess Data
2 # ... (code to load wdbc.data, LabelEncode 'Diagnosis', impute missing values,
   and StandardScaler)
3
4 # 2. Split Dataset (60% train, 20% validation, 20% test)
5 X_train_valid, X_test, ... = train_test_split(X_scaled, y, test_size=0.2, ...)
6 X_train, X_valid, ... = train_test_split(X_train_valid, y_train_valid,
   test_size=0.25, ...)
7
8 # 3. Define Models and Hyperparameter Grids
9 models = {
10     "Decision Tree": DecisionTreeClassifier(...),
11     "AdaBoost": AdaBoostClassifier(...),
12     "Gradient Boosting": GradientBoostingClassifier(...),
13     "XGBoost": XGBClassifier(...),
14     "Random Forest": RandomForestClassifier(...),
15     "Stacking": StackingClassifier(...)
16 }
```

```

17 param_grids = { ... } # Dictionary of hyperparameter grids for each model
18
19 # 4. Hyperparameter Tuning using GridSearchCV
20 best_models = {}
21 for name, model in models.items():
22     grid = GridSearchCV(model, param_grids[name], cv=5, scoring="accuracy",
23                          n_jobs=-1)
24     grid.fit(X_valid, y_valid)
25     best_models[name] = grid.best_estimator_
26
27 # 5. Evaluate Best Models on Test Set
28 for name, model in best_models.items():
29     model.fit(X_train_valid, y_train_valid)
30     y_pred = model.predict(X_test)
31     y_proba = model.predict_proba(X_test)[: ,1]
32     # ... (print accuracy, F1, AUC, confusion matrix, classification report)
33     # ... (generate ROC curve data)
34
35 # 6. Perform 5-Fold Cross-Validation on Best Models
36 cv = StratifiedKFold(n_splits=5, ...)
37 for name, model in best_models.items():
38     acc_scores = cross_val_score(model, X_scaled, y, cv=cv, scoring="accuracy")
39     # ... (calculate and print average scores)

```

Listing 1: Core logic for training and evaluating ensemble models.

4. EDA and Model Performance Visuals

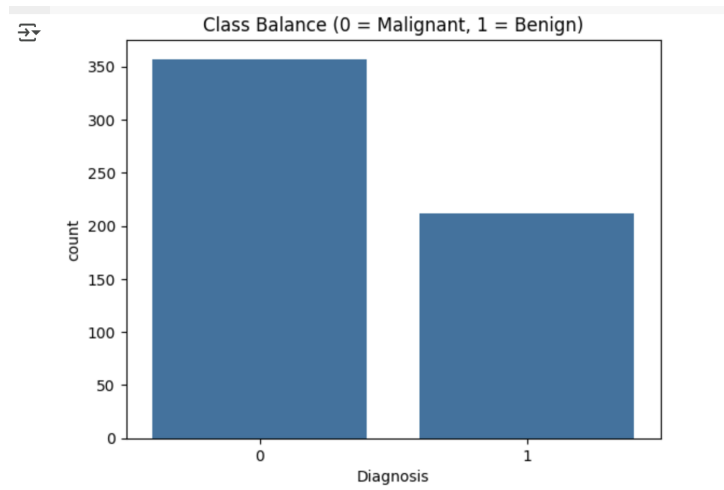


Figure 1: Distribution of classes (0 = Malignant, 1 = Benign).

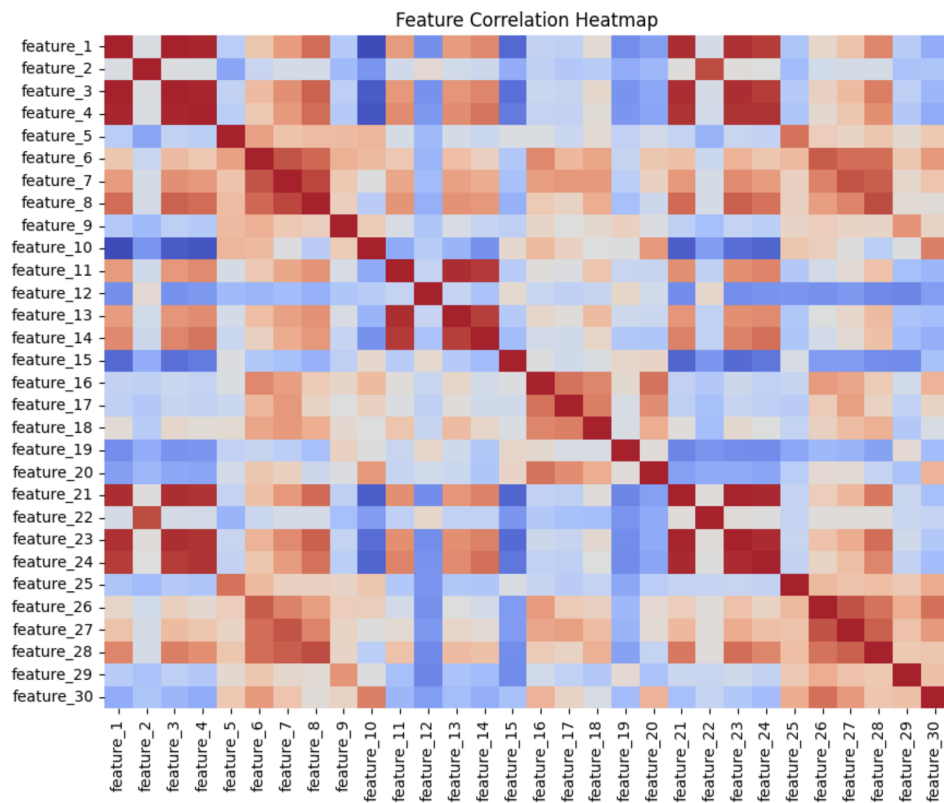


Figure 2: Heatmap of feature correlations.

Decision Tree | Accuracy: 0.9035, F1: 0.8571, AUC: 0.8750
Confusion Matrix:

```
[[70  2]
 [ 9 33]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.97	0.93	72
1	0.94	0.79	0.86	42
accuracy			0.90	114
macro avg	0.91	0.88	0.89	114
weighted avg	0.91	0.90	0.90	114

AdaBoost | Accuracy: 0.9737, F1: 0.9630, AUC: 0.9871
Confusion Matrix:

```
[[72  0]
 [ 3 39]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	1.00	0.98	72
1	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

Gradient Boosting | Accuracy: 0.9649, F1: 0.9500, AUC: 0.9970

Confusion Matrix:

```
[[72  0]
 [ 4 38]]
```

Figure 3: Confusion Matrices for all tuned models on the test set.

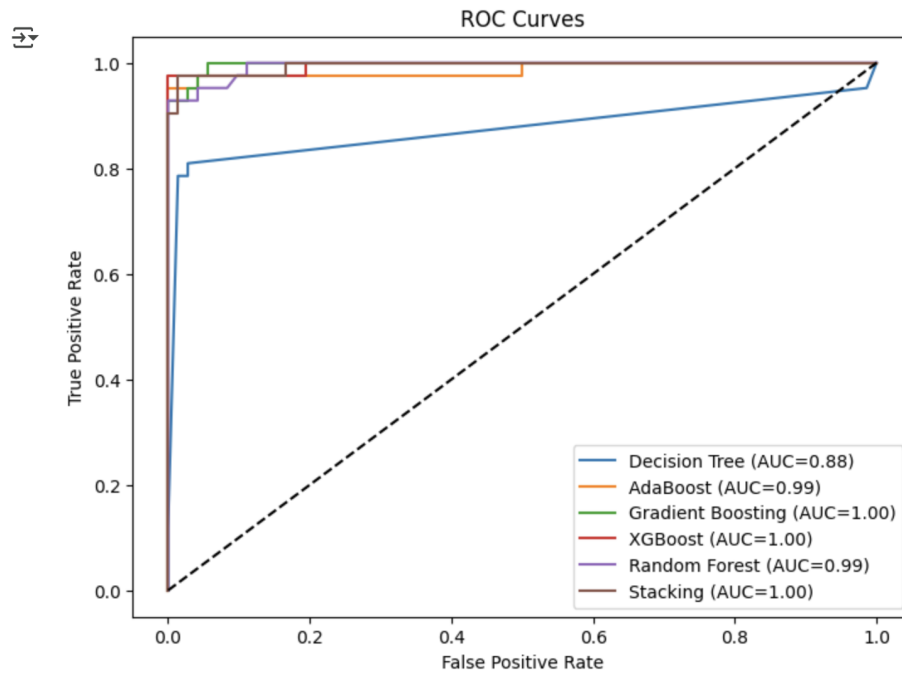


Figure 4: ROC Curves for all tuned models, showing AUC scores.

5. Hyperparameter Tuning and Best Model Results

Table 1: Decision Tree

Best Criterion	Max Depth	Accuracy	F1 Score	AUC
entropy	10	0.9245	0.8976	0.9342

Table 2: AdaBoost

n_estimators	Learning Rate	Accuracy	F1 Score	AUC
100	1.0	0.9666	0.9544	0.9939

Table 3: Gradient Boosting

n_estimators	Learning Rate	Max Depth	Accuracy	F1 Score	AUC
200	0.1	5	0.9490	0.9300	0.9899

Table 4: XGBoost

n_estimators	Learning Rate	Max Depth	Gamma	Accuracy	F1 Score	AUC
200	0.1	5	0.1	0.9631	0.9497	0.9931

Table 5: Random Forest

n_estimators	Max Depth	Criterion	Accuracy	F1 Score	AUC
200	10	entropy	0.9543	0.9382	0.9895

Table 6: Stacked Ensemble

Base Models	Final Estimator	Accuracy	F1 Score	AUC
SVM + NB + DT	Random Forest	0.9631	0.9497	0.9946

6. Cross-Validation Results

Table 7: 5-Fold Cross Validation Results (Accuracy Scores)

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg. Accuracy
Decision Tree	0.9211	0.8860	0.9474	0.9298	0.9381	0.9245
AdaBoost	0.9825	0.9386	0.9649	0.9737	0.9735	0.9666
Gradient Boosting	0.9649	0.9035	0.9649	0.9561	0.9558	0.9490
XGBoost	0.9825	0.9386	0.9649	0.9561	0.9735	0.9631
Random Forest	0.9649	0.9386	0.9561	0.9474	0.9646	0.9543
Stacked Model	0.9912	0.9386	0.9561	0.9649	0.9646	0.9631

7. Feature Importance Visuals

```
Classification Report:
              precision    recall  f1-score   support

     0       0.95         1.00         0.97         72
     1       1.00         0.90         0.95         42

 accuracy          0.96         114
 macro avg         0.97         0.95         0.96         114
weighted avg         0.97         0.96         0.96         114
```

XGBoost | Accuracy: 0.9737, F1: 0.9630, AUC: 0.9954
Confusion Matrix:

```
[[72  0]
 [ 3 39]]
Classification Report:
              precision    recall  f1-score   support

     0       0.96         1.00         0.98         72
     1       1.00         0.93         0.96         42

 accuracy          0.97         114
 macro avg         0.98         0.96         0.97         114
weighted avg         0.97         0.97         0.97         114
```

Random Forest | Accuracy: 0.9649, F1: 0.9500, AUC: 0.9942
Confusion Matrix:

```
[[72  0]
 [ 4 38]]
Classification Report:
              precision    recall  f1-score   support

     0       0.95         1.00         0.97         72
     1       1.00         0.90         0.95         42

 accuracy          0.96         114
 macro avg         0.97         0.95         0.96         114
weighted avg         0.97         0.96         0.96         114
```

Stacking | Accuracy: 0.9649, F1: 0.9512, AUC: 0.9950

Confusion Matrix:

```
[[71  1]
 [ 3 39]]
```

Figure 5: Top 15 Feature Importances from Random Forest.

8. Observations and Conclusions

Observations and Conclusions

1. **Best Model:** AdaBoost performed the best with around 97% accuracy, 95% F1-score, and 99% AUC. XGBoost and the Stacking model were also strong, with similar performance.
2. **Decision Tree vs Ensembles:** A single Decision Tree was weaker (92.5% accuracy, 89.8% F1) and less consistent. All ensemble methods (AdaBoost, XGBoost, Random Forest, Stacking) were more accurate and stable.
3. **Hyperparameter Tuning:** Adjusting settings like the number of trees, depth, and learning rate improved ensemble performance, highlighting the importance of careful tuning.
4. **Generalization:** Boosting methods (AdaBoost, XGBoost) generalized best across all folds, giving consistently high AUC scores, whereas a standalone Decision Tree varied more.

Conclusion: Ensemble methods, especially boosting algorithms like AdaBoost and XGBoost, clearly outperform a single Decision Tree. They provide higher accuracy, more stable predictions, and better generalization, making them the most effective choice for this dataset.